TECHNICAL REPORT · ARCHITECTURE, IMPLEMENTATION & EVALUATION

Sri Lanka Share Market Autonomous News Analysis Agent

A Hybrid Neuro-Symbolic Multi-Step AI Agent for the Colombo Stock Exchange (CSE)

PROJECT TYPE	Autonomous AI / LLM Engineering Group Project
MARKET	Colombo Stock Exchange (CSE), Sri Lanka
ORCHESTRATION	LangGraph StateGraph (src/agent.py) & Sequential Pipeline (src/pipeline.py)
CORE STACK	Python 3.11+, Pydantic v2, SQLite, Trafilatura, RapidFuzz, Gemini / Claude / OpenRouter
CONTRIBUTORS	Indrajith Deegalla | Akila Kasun | Prasanna
DATE	October 2026

ABSTRACT & EXECUTIVE SUMMARY

In one paragraph: Retail investors and analysts following the Colombo Stock Exchange (CSE) must read multiple financial news websites daily, where the same announcements are repeated across outlets and mixed with non-financial noise. At the same time, attempting to predict exact share price movements in a small, emerging market using machine learning is statistically unreliable due to small sample sizes. This technical report presents the design, implementation, and evaluation of the **CSE News Analysis Agent**—an autonomous five-step hybrid AI system orchestrated with LangGraph. The agent collects Sri Lankan financial RSS feeds and CSE end-of-day trading summaries, strips HTML and extracts full article prose via *trafilatura*, merges near-duplicate stories across outlets using *rapidfuzz*, classifies each unique story into one of 16 CSE-aligned sector buckets via an LLM constrained by a strict Pydantic schema, aggregates sector scores using deterministic weighted arithmetic, and produces a daily Markdown brief complete with pro/con evidence and calibrated confidence ratings.

1. Problem Statement and Scenario

1.1 The Operational Scenario

An investor, analyst, or researcher monitoring the Colombo Stock Exchange (CSE) must track developments across roughly 290 listed companies spanning banking, manufacturing, plantations, hospitality, energy, and diversified conglomerates, alongside macroeconomic policy announcements from the Central Bank of Sri Lanka (CBSL), the International Monetary Fund (IMF), and the Ministry of Finance. Every trading day, dozens of business articles are published across Sri Lankan financial portals such as *EconomyNext*, *Daily FT*, and *Daily Mirror*.

In daily practice, human market participants encounter three acute information bottlenecks:

- **Fragmentation and Cross-Outlet Redundancy:** A single corporate event—such as a commercial bank's quarterly earnings release, a debenture oversubscription, or a sovereign debt update—is routinely published by four or five websites under rephrased headlines. Reading and mentally deduplicating these stories wastes time and risks counting the same catalyst multiple times.
- **Low Signal-to-Noise Ratio:** Financial RSS feeds interleave material corporate disclosures (e.g., CSE Watch List placements, public procurement contracts, power grid blackouts) with ceremonial corporate awards, routine administrative notices, and unrelated political or sports stories.
- **Cognitive Fatigue and Confirmation Bias:** A human scanning six websites every morning can easily miss a critical regulatory announcement or selectively focus on positive headlines while overlooking structural downside risks.

1.2 Why Sector-Level Evidence Instead of Share Price Prediction?

A common mistake in financial AI projects is attempting to build an end-to-end share price predictor (e.g., *'Stock X will rise 4.2% tomorrow'*). We deliberately scoped our system to reject price prediction and company-level entity linking in Phase 1 for two fundamental reasons:

WHY WE PRESENT SECTOR EVIDENCE AND NEVER PREDICT PRICES

1. Small-Sample Sparsity: The Sri Lankan equity market is small. For any one type of corporate or macroeconomic news event, historical archives may contain only 5 to 15 past examples—far too few to train a model that predicts price movements reliably.

2. Single-Stock Illiquidity & Entity Ambiguity: Many CSE counters trade infrequently, so a single share's price moves randomly on thin volume. Furthermore, resolving exact company names from free text is brittle because the same firm is written in many ways (*'Sampath'*, *'Sampath Bank PLC'*, or ticker *'SAMP.N0000'*). Working at the **sector level** (16 CSE-aligned buckets) averages out random single-stock noise and improves classification reliability.

Our Rule: A tool that promises exact price numbers will be wrong often and lose user trust within weeks. Our system presents structured evidence—arguments in favour, arguments against, and confidence—while the human makes the decision.

2. System Architecture

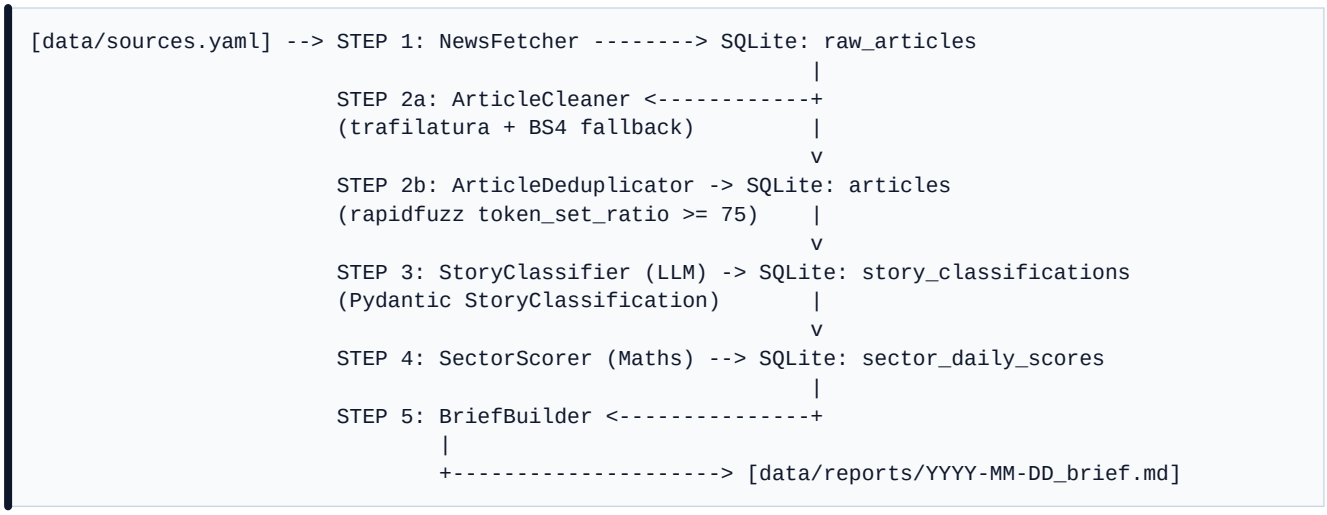
2.1 Hybrid Neuro-Symbolic Division of Labour

The central architectural principle of the CSE News Analysis Agent is a strict separation of responsibilities between **deterministic Python code** and the **Large Language Model (LLM)**. In many naive agent architectures, the LLM is asked to scrape, deduplicate, compute scores, and format reports—leading to non-deterministic bugs and arithmetic hallucinations. In our design, **the AI never does maths and never decides control flow**:

Component	Module Path	Responsibility	Uses AI?
News Collector	src/collect/news_fetcher.py	Reads RSS feeds from data/sources.yaml via feedparser; inserts unique URLs into raw_articles.	No
Price Collector	src/collect/price_fetcher.py	Polls CSE internal POST endpoints for ASPI, S&P; SL20, and closing equity trades; saves JSON snapshots.	No
Sector Mapper	src/collect/sector_mapper.py	Maps CSE company symbols and names into 14 canonical sectors and generates data/sector_map.csv.	No
Article Cleaner	src/process/cleaner.py	Extracts full article text using trafilatura, with BeautifulSoup HTML stripping fallback on RSS summaries.	No
Deduplicator	src/process/deduplicator.py	Groups near-duplicate stories across news portals using rapidfuzz token-set similarity (>= 75.0).	No
AI Classifier	src/analyse/classifier.py	Classifies each unique story into sector, sentiment [-1,+1], importance, novelty, and pro/con reasons.	**Yes (LLM)**
Sector Scorer	src/analyse/scorer.py	Computes importance-weighted sector scores, discounts repeat news, and calibrates confidence via plain maths.	No
Brief Builder	src/report/brief_builder.py	Compiles the daily Markdown report (data/reports/YYYY-MM-DD_brief.md) and Rich terminal summary.	No

2.2 End-to-End Pipeline and Relational Database Schema (src/db.py)

All stages communicate through a local SQLite database (**data/cse.db**) comprising seven tables: **raw_articles** (unprocessed RSS entries with UNIQUE url constraint), **articles** (cleaned, deduplicated canonical stories with story_hash and duplicate_count), **story_classifications** (validated LLM structured outputs per article_id), **sector_daily_scores** (daily aggregated scores, pro/con arguments, and confidence per sector), **daily_market** (daily ASPI, S&P SL20, market turnover, and market cap), **daily_prices** (per-company closing price, change %, share volume, and turnover), and **fetch_log** (operational audit log for every collection run).



3. Agents, Tools, and the Agent Loop

3.1 Dual Orchestration: Plain Pipeline vs. LangGraph State Machine

The system provides two execution entry points that coordinate the five pipeline tools:

- **Sequential Function Runner (src/pipeline.py):** Invokes Steps 1 to 5 in a deterministic straight line (*run_daily_pipeline()*) with interactive Rich console progress bars and preview tables.
- **LangGraph State Machine Orchestrator (src/agent.py):** Models the agent loop as a compiled *StateGraph(AgentState)* with five nodes (*collect*, *process*, *classify*, *score*, and *report*) operating over a shared, strongly typed state dictionary.

```
class AgentState(TypedDict):
    db_path: Path
    fetch_online: bool
    trade_date: str
    new_articles_count: int
    cleaned_count: int
    unique_stories_count: int
    classified_count: int
    sectors_scored: int
    report_path: str
    errors: List[str]

# Graph Topology in build_cse_agent_graph():
# collect --> process --> classify --> score --> report --> END
```

3.2 Agent Loop Execution and Fault Isolation

When *run_agent()* is invoked, the database schema is initialized, the current Sri Lanka trading date (*UTC+05:30*) is resolved, and the initial *AgentState* is passed into the compiled graph. Each node wraps its underlying tool class in a fault-isolating *try...except* block that appends any runtime exceptions to *state['errors']* rather than crashing the pipeline. Consequently, if a single RSS feed times out during *node_collect_news*, downstream nodes still clean, deduplicate, classify, and score all valid articles already persisted in SQLite. A conditional routing policy (*decide_next_step*) monitors error accumulation across transitions.

3.3 Core Tool Implementations

- **NewsFetcher (src/collect/news_fetcher.py):** Parses RSS feeds from *EconomyNext*, *Daily FT (Business & Financial Services)*, and *Daily Mirror (Business)*. Uses *INSERT OR IGNORE INTO raw_articles* keyed on *UNIQUE(url)* so repeated runs never duplicate raw entries.
- **PriceFetcher & SectorMapper (src/collect/):** Sends POST requests to CSE's internal endpoints (*dailyMarketSummery* and *detailedTrades*), saves raw JSON audit files in *data/snapshots/*, and records per-company closing prices. *SectorMapper* maps all listed symbols into the 14 CSE sectors using explicit ticker overrides (*TICKER_SECTOR_OVERRIDES*) and ordered keyword rules (*KEYWORD_RULES*).
- **ArticleCleaner (src/process/cleaner.py):** Attempts full-page prose extraction via *trafilatura.extract()*. If blocked or under 40 characters, falls back to *strip_html_fallback()* using BeautifulSoup (*lxml*) to decompose scripts/styles/menus from the RSS summary, and finally to *title_only*.
- **ArticleDeduplicator (src/process/deduplicator.py):** Normalizes titles by stripping trailing publisher names (e.g., *'- EconomyNext'*, *'| Daily FT'*) and punctuation, then computes a weighted fuzzy similarity score against recent canonical stories: **0.70 * TokenSetRatio(Title) + 0.30 * TokenSetRatio(Lead300Chars)**. Stories scoring **>= 75.0** are merged into the canonical record, incrementing *duplicate_count* and keeping the longer *clean_text*.
- **SectorScorer (src/analyse/scorer.py):** Aggregates story scores per sector using deterministic importance weights (*trivial = 0.5*, *moderate = 1.0*, *major = 2.0*). If a story is marked as a 7-day repeat (*is_new = False*), its weight is multiplied by **0.25** (a 75% discount). Sector confidence is averaged numerically (*LOW=1, MEDIUM=2, HIGH=3*) and strictly capped at **MEDIUM** whenever *article_count == 1*.

- **BriefBuilder (src/report/brief_builder.py):** Generates the daily Markdown brief at *data/reports/YYYY-MM-DD_brief.md* featuring the legal disclaimer, ranked sector evidence blocks (*In favour*, *Against*, *How sure*, *Based on N stories*), and the top 15 market stories.

4. Prompt Design and Structured Outputs

4.1 System Prompt Engineering and Temporal Context Injection

In **src/analyse/classifier.py**, the system prompt enforces eight strict rules designed to prevent speculative investment advice, enforce conciseness, and mandate dialectical reasoning (requiring both a supporting reason and an opposing counter-argument for every story):

```
SYSTEM_PROMPT = """You are an objective financial news analyst for the
Colombo Stock Exchange (CSE) in Sri Lanka.
Classify the news story strictly into the requested JSON schema.

Rules:
1. NEVER give investment advice, never say 'buy' or 'sell', and never
   predict share price percentages.
2. Assign the single best CSE sector. Use 'Market-Wide' for Central Bank
   (CBSL) interest rates, inflation, IMF, taxes, or LKR exchange rates.
   Use 'Unrelated' for non-business news.
3. Score sentiment from -1.0 (very negative) to +1.0 (very positive).
4. Set is_new=False if the story repeats an event in RECENT HEADLINES.
5. 'reasoning' MUST be ONE sentence and <= 200 characters.
6. 'counter_argument' MUST be ONE sentence and <= 200 characters.
7. Never exceed these character limits. Be concise.
8. Set confidence to 'LOW', 'MEDIUM', or 'HIGH'."""
```

Before classifying each story, *get_recent_titles()* queries SQLite for up to 8 classified headlines from the preceding 7 days and prepends them to the user prompt under *RECENT HEADLINES (last 7 days)*, followed by the target headline and a 1,500-character body snippet. This allows the LLM to accurately set *is_new = False* when an ongoing story is re-reported later in the week.

4.2 Pydantic Structured Output Schema (src/analyse/schema.py)

All LLM responses are validated against the **StoryClassification** Pydantic model, which supports three interchangeable backends (Google Gemini *gemini-2.5-flash*, Anthropic *claude-3-5-haiku-20241022*, and OpenRouter *nvidia/nemotron-3.5-lightning:free*) at *temperature=0.0*:

- **sector (SectorType):** Restricted via *Literal[...]* to 16 buckets: *Banks, Diversified Financials, Insurance, Hotels & Travels, Construction & Engineering, Manufacturing & Materials, Food, Beverage & Tobacco, Plantations, Telecommunications & IT, Energy & Utilities, Healthcare, Retails & Consumer, Real Estate, Transportation & Logistics, Market-Wide, Unrelated*.
- **sentiment_score (float):** Constrained to *[-1.0, +1.0]* via *Field(ge=-1.0, le=1.0)* and rounded to 2 decimal places by *@field_validator('sentiment_score')*.
- **importance & confidence:** Restricted to *Literal['trivial', 'moderate', 'major']* and *Literal['LOW', 'MEDIUM', 'HIGH']*.
- **reasoning & counter_argument:** Enforced *min_length=5, max_length=220* strings providing a one-sentence justification and a one-sentence caveat or opposing risk.

5. Testing and Verification

To verify correctness offline without incurring API costs or relying on external network availability, the repository includes 8 unit test suites under **tests/** executed via *pytest*:

Test Suite	What It Verifies
test_news_fetcher.py	Mocks feedparser.parse; verifies first run inserts 2 articles and logs to fetch_log, while second run inserts 0 duplicates.
test_price_fetcher.py	Verifies Colombo midnight boundary timestamp parsing (1754505000000 ms -> 2025-08-07 in UTC+05:30), DB upserts, and JSON snapshots.
test_sector_mapper.py	Verifies deterministic company classification (COMB -> Banks, DIAL -> Telecom & IT, CITW -> Hotels & Travels) and CSV export.
test_cleaner.py	Verifies <script> tag stripping in strip_html_fallback() and seamless fallback from trafilatura_full to rss_summary_fallback.
test_deduplicator.py	Verifies cross-outlet duplicate matching (>= 75.0), canonical story merging (duplicate_count=2), longest-text retention, and idempotency.
test_classifier.py	Verifies Pydantic score rounding (0.756 -> 0.76), ValidationError on out-of-bounds scores (+2.5), and idempotent DB classification.
test_scorer.py	Verifies weighted score maths (0.60 and 0.67), single-article confidence capping at MEDIUM, and filtering of Unrelated stories.
test_brief_builder.py	Verifies exact Figure 2 sector evidence block formatting and presence of the mandatory legal disclaimer in generated Markdown files.

6. Challenges, Limitations, and Future Improvements

6.1 Technical Challenges Resolved

- **Colombo Midnight Timestamp Shift (UTC vs. UTC+05:30):** The CSE API returns *tradeDate* as epoch milliseconds at 00:00:00 Sri Lanka time. Standard UTC conversion shifts midnight back by 5.5 hours into the previous date. We fixed this in *parse_cse_timestamp()* by explicitly binding *SL_TZ = timezone(timedelta(hours=5, minutes=30))*.
- **Unofficial CSE API Nesting & Schema Drift:** CSE's internal endpoints wrap dictionary payloads inside nested lists (*[[{ ... }]]*). We implemented defensive list-unwrapping loops and raw JSON snapshot archiving (*data/snapshots/*) so raw trading data is never lost.
- **LLM Rate Limits, Markdown Fences, and Native DLL Conflicts:** Free-tier APIs frequently return transient HTTP 429/503 errors or wrap JSON output in Markdown fences (```*json*). We built lazy client initialization, direct *httpx* REST support for Anthropic and OpenRouter, fence sanitization, and linear backoff retries (*base_delay * (attempt + 1)*).

6.2 Current Limitations

- **English-Only Coverage in Phase 1:** The current RSS collector processes English financial publications; Sinhala and Tamil news sources are not yet translated.
- **Multi-Sector Conglomerates:** Large CSE holding groups (e.g., JKH, Hayleys) operate across several industries, so assigning a single primary sector to a parent-level announcement can simplify cross-sector impacts.
- **Full-Text Paywalls / Anti-Bot Blocks:** When a site blocks automated scraping, *ArticleCleaner* falls back to the shorter RSS summary.

6.3 Future Improvements (Phases 2 and 3)

- **Phase 2 — Historical Event-to-Price Lookup:** Once 3+ months of daily closing prices have accumulated in *daily_prices*, the system will join historical sector sentiment spikes with subsequent 3-day sector returns to report empirical base rates (e.g., *'This type of news happened 11 times before; 8 times the sector moved 2-5% within 3 days'*).
- **Phase 3 — Quarterly Financial PDF Verification:** Parsing official CSE quarterly earnings PDFs to cross-check media sentiment against audited revenue and profit numbers, warning users when news mood and financial fundamentals disagree.

7. Individual Contributions

The CSE News Analysis Agent was developed collaboratively across data engineering, natural language processing, and agentic orchestration workstreams. Individual responsibilities and module ownership are detailed below:

1. Akila Kasun

Primary Role & Focus Area: Data Collection, Fuzzy Deduplication Engine & Frontend Terminal UI

Modules Owned: *src/collect/news_fetcher.py, src/collect/price_fetcher.py, src/collect/sector_mapper.py, src/process/deduplicator.py, data/sources.yaml*

Contribution Summary: Engineered data collection modules (RSS news and CSE price fetchers), developed the fuzzy deduplication engine using rapidfuzz, and designed frontend terminal UI visualization elements.

Key Technical Contributions:

- Engineered the RSS financial news collector (news_fetcher.py) and CSE daily closing price collector (price_fetcher.py) with UTC+05:30 timestamp handling and raw JSON audit snapshots.
- Developed the cross-outlet fuzzy deduplication engine (deduplicator.py) utilizing rapidfuzz token-set similarity (70% title / 30% lead text weighting) to merge repeated stories.
- Designed the Rich frontend terminal UI visualization elements, including progress spinners, color-coded mood panels, and structured summary tables across pipeline stages.

2. Prasanna

Primary Role & Focus Area: Database Architecture, HTML Extraction & Daily Report Generation

Modules Owned: *src/db.py, src/process/cleaner.py, src/report/brief_builder.py, src/pipeline.py*

Contribution Summary: Built the database schema initialization layer (db.py), developed the HTML cleaner module utilizing trafilatura and BeautifulSoup, and authored the reporting generator (brief_builder.py).

Key Technical Contributions:

- Built the SQLite relational database schema and connection initialization layer (db.py) spanning all 7 operational tables and audit logs.
- Developed the multi-stage article cleaner module (cleaner.py) utilizing trafilatura for full-webpage prose extraction with a BeautifulSoup (lxml) HTML stripping fallback.
- Authored the daily report generator (brief_builder.py) to compile ranked sector evidence blocks (In favour, Against, Confidence) and top market stories into Markdown briefs.

3. Indrajith Deegalla

Primary Role & Focus Area: AI Story Classifier, Sector Scoring Engine, LangGraph Orchestrator & Test Suite

Modules Owned: *src/analyse/schema.py, src/analyse/classifier.py, src/analyse/scorer.py, src/agent.py, tests/**

Contribution Summary: Implemented the multi-provider AI story classifier and Pydantic validation schemas, engineered the deterministic sector scoring engine, structured the LangGraph state machine orchestrator (agent.py), and designed the comprehensive offline test suite.

Key Technical Contributions:

- Implemented the multi-provider LLM story classifier (classifier.py) supporting Gemini, Anthropic Claude, and OpenRouter alongside strict Pydantic validation schemas (schema.py).
- Engineered the deterministic mathematical sector scoring engine (scorer.py) with importance weighting, 7-day repeat story discounting, and single-article confidence capping.
- Structured the autonomous LangGraph state machine orchestrator (agent.py) with per-node fault isolation and designed the comprehensive 8-module offline pytest suite (tests/).